RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

UNIT-V (Exception Handling)

- Errors
- Exceptions
- Checked and Unchecked Exceptions
- Control Flow in Exceptions
- try and catch block
- Multiple catch block
- Nested try
- Finally block
- throw keyword

Error

- Errors that occur at program runtime can seriously interrupt the normal flow of a program.

- Some common causes of errors are-

➢division by 0, or values that are too large or small for a type

➢no memory available for dynamic allocation

➢errors on file access, for example, file not found

➢attempt to access an invalid address in main memory

➢invalid user input

Error

- Anomalies like these lead to incorrect results and may cause a computer to crash.

- Both of these cases can have fatal effects on your application.

- **There are three types of errors in programming.**

➢Syntax

➢Semantic

➢Logical

Syntax Error

- Syntax is the structure of statements in a program.
- A syntax error is an error in the syntax of a programming language, i.e., an error in the source code of a program.
- If syntax errors are present in the program then the compilation of the program fails and is terminated after showing the list of errors and the line number where the errors have occurred.
- In some cases the line number may not exactly indicate the correct place of the error.
- In some other cases, a single syntax error can result in a long list of errors.

Semantic Error

- A semantic occurs due to the wrong use of the variables.
- The syntax of the programming language would be correct but the problem may be caused due to the use of wrong variables, wrong operations, or in a wrong order.
- Semantics errors are not traced by the compiler, as a result of which, the program executes but does not return the desired result.
- Semantic errors occur during the execution of the code.
- E.g. Forgetting to divide by 100 when calculating a percentage amount.

Semantic Error

```
#include <iostream>
Using namespace std;
int main()
{
   int x = 5;
    int y = 0;
    int result = x / y; // Division by zero
    cout << "Result: " << result << std::endl;

    return 0;
}
```

Logic Error

- As the name itself implies, these errors are related to the logic of the program.
- Logical error is a mistake in a program's source code.
- Logical errors are also not detected by the compiler and cause incorrect results.
- To trace a logical error, the programmer must check the logic/algorithm of the program.
- E.g. Infinite loop

```
a=10
while (a>0)
{ }
```

Conventional Error Handling Methods

- Methods of handling errors usually involve returning an error code from every function.
- Either through a return value, an extra reference parameter, or a global variable such as ‘errno’.
- Exceptions are a brilliant programmatic concept, but implementation of them in C++ language leaves a lot to be done.

Conventional Error Handling Methods

- These methods of error reporting, however, have several limitations:

➢Checking for an error condition after every function call practically doubles the amount of code you have to write.

➢Programmer may forget to check for an error condition, or otherwise choose to ignore them.

➢Propagating errors upward from function to function can become untidy because of the constraint of returning one type of error condition from a function.

Exception

- Exceptions are run time anomalies or unusual conditions that a program may encounter during execution.

- It was not a part of original C++.

- It is a new feature added to ANSI C++.

Exception

- Exceptions are of 2 kinds:
  - Synchronous Exception:
    - Out of range index
    - Over flow
  - Asynchronous Exception: Error that are caused by events beyond the control of the program
    - Keyboard interrupts
- In C++ only synchronous exceptions can be handled.

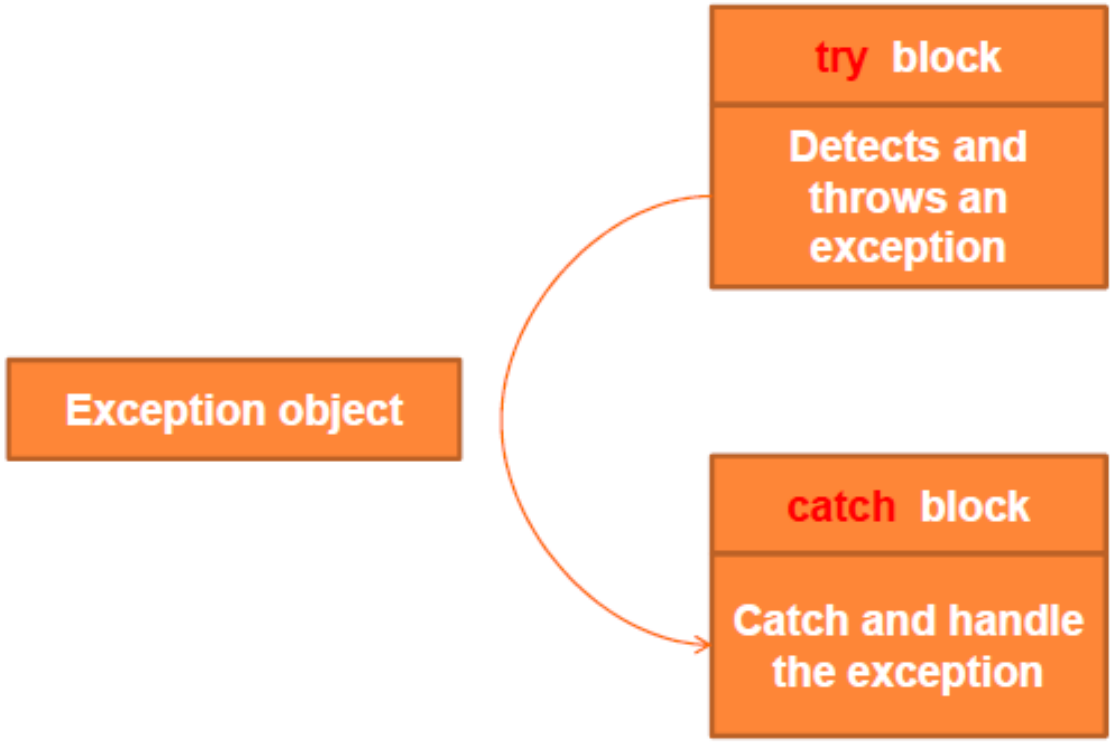
Exception Handling

- **Exception handling**
  - Can resolve exceptions
    - Allow a program to continue executing or
    - Notify the user of the problem and
    - Terminate the program in a controlled manner
  - **Makes programs robust and fault-tolerant**
- **Exception handling mechanism**
  - Find the problem (Hit the exception)
  - Inform that an error has occurred (Throw the exception)
  - Receive the error information (Catch the exception)
  - Take corrective action (handle the exception)

Exception handling mechanism

- It is basically build upon three keywords
  - Try
  - Throw
  - Catch

C++ Exceptions



Exception handling mechanism

- The keyword **try** is used to preface a block of statements which may generate exceptions.
- When an exception is detected, it is thrown using a **throw** statement in the try block.
- A **catch** block defined by the keyword 'catch' catches the exception and handles it appropriately.
- The catch block that catches an exception must immediately follow the try block that throws the exception. (No other code in between)

C++ Exception handling mechanism

- **try {…} Block**
  defines, for thrown exceptions, an enclosing context with specified catch handlers

- **throw:**

constructs an exception object, e, and takes it out of an enclosing context defined by a try block

- catch(E e) {…} Block
  exception handler catch(E e) responds to an exception object of type "E"

Exception handling mechanism

```
try
{
   …                          //Block of Statements which detects and
  throw exception;          // throw an exception
   …
}
catch(type arg)             //catch exception
{
…                             //Block of statements that handles exception
…
}
….
….
```

Example

```
#include <iostream>
using namespace std;
 int main()
{
int x = -1;
cout << "Before try \n";
 try
{
      cout << "Inside try \n";
      if (x < 0)
      {
          throw x;     //throw an exception
          cout << "After throw (Never executed) \n";
      }
   }
  catch (int x ) {
      cout << "Exception Caught \n"; }
    cout << "After catch (Will be executed) \n";
   return 0;
}
```

Output

```
Before try
Inside try
Exception Caught
After catch (Will be executed)
```

Exception Handling

- Enclose code that may have an error in try block.
- Follow with one or more catch blocks. Each catch block has an exception handler.
- If exception occurs and matches parameter in catch block, code in catch block executed.
- If no exception thrown, exception handlers skipped and control resumes after catch blocks.
- throw point - place where exception occurred Control cannot return to throw point.

Exception Handling

- Exceptions are objects used to transmit information about a problem.

- If the type of the object thrown matches the arg type in the catch statement, the catch block is executed.

Example

```
#include<iostream>
using namespace std;
int main()
{
int a,b;
cout<<"Enter the values of a and b";
cin>>a>>b;
int x = a- b;
try
{
if(x!=0)
{cout<<"result(a/x) = "<<a/x<<"\n";
}
else{
throw(x);}}
catch(int i)
{
cout<<"exception caught :
x =  "<<x<<"\n";
}
cout<<"End";
return 0;
}
```

Output

First Run Output:
Enter the values of a and b
20 15
Result(a/x) =4
End
Second Run Output:
Enter the values of a and b
10 10
exception caught : x =0
End

Throwing Mechanism

- The **throw** statement can have one of the following 3 forms
  - throw(exception)
  - throw exception
  - throw //used to re-throw a exception

- The operand object exception can be of any type, including **constant**.

- It is also possible to throw an object not intended for error handling.

Throwing MECHANISM

- Throw point can be in a deeply nested scope within a try block or in a deeply nested function call.

- In any case, control is transferred to the catch statement.

Catching Mechanism

- The type indicates the type of exception the catch block handles.
- The parameter arg is an **optional** parameter name.
- The catch statement catches an exception whose type matches with the type of the catch argument

```
catch(type arg)
{
…
…
}
```

Catching Mechanism

- If the parameter in the catch statement is named, then the parameter can be used in the exception handling code.

- If a catch statement does not match the exception it is skipped.

- More than one catch statement can be associated with a try block.

Catching Mechanism

- When an exception is thrown, the exception handlers are searched **in order** for a match.
- The first handler that yields a match is executed.
- If several catch statement matches the type of an exception, the first handler that matches the exception type is executed.

Multiple catch statements

```
try {
// try block
}
catch (type1 arg) {
// catch block
}
catch (type2 arg) {
// catch block
}
catch (type3 arg) {
// catch block
}
…
…
```

```
catch (typeN arg) {
// catch block}
```

Example(Multiple catches)

```
#include<iostream>
using namespace std;
int main()
{
int a=10, b=0, c;
try
{
    if(b==0)
    { throw(c);}
     else
     {c=a/b;}
 }
catch(char c)
{
cout<<"Caught exception : char type ";
}
catch(int i)
{
cout<<"Caught exception : int type ";
}
catch(short s)
{
 cout<<"Caught exception : short type ";
}
cout<<"\n Hello";
}
```

Output

```
Caught exception : int type
 Hello
```

Catching All Exceptions

- In some circumstances you will want an exception handler to catch all exceptions instead of just a certain type.

- Simply use this form of **catch**.

```
catch(...)
{
// process all exceptions
}
```

Catching All Exceptions

```
// This example catches all exceptions.
#include <iostream>
using namespace std;
void Xhandler(int test)
{
try{
if(test==0) throw test;
if(test==1) throw 'a';
if(test==2) throw 123.23;
}
catch(...)
{ // catch all exceptions
cout << "Caught One!\n";
}
}
```

```
int main()
{
cout << "Start\n";
Xhandler(0);
Xhandler(1);
Xhandler(2);
cout << "End";
return 0;
}
```

Output

```
Start
Caught One!
Caught One!
Caught One!
End
```

Exception Handling using function call

- Often, Exceptions are thrown by functions that are invoked from within the try blocks.
- The point at which the throw is executed is called the **throw point**.
- Once an exception is thrown to the catch block, control cannot return to the throw point.

Example

```
#include<iostream>
using namespace std;
void divide(int x, int y, int z)
{
      cout<<"inside the function\n";
       if((x-y)!=0)
         {
           int  R=z/(x-y);
           cout<<"Result="<<R;}
       else
        {
        throw(x-y);
        }
}
```

```
int main()
{
  try
   {
      cout<<"inside try block\n";
      divide(10,20,30);
      divide(10,10,20);
      }
    catch(int i)              //catches exception
    {
     cout<<"\ncaught exception";
    }
return 0;
}
```

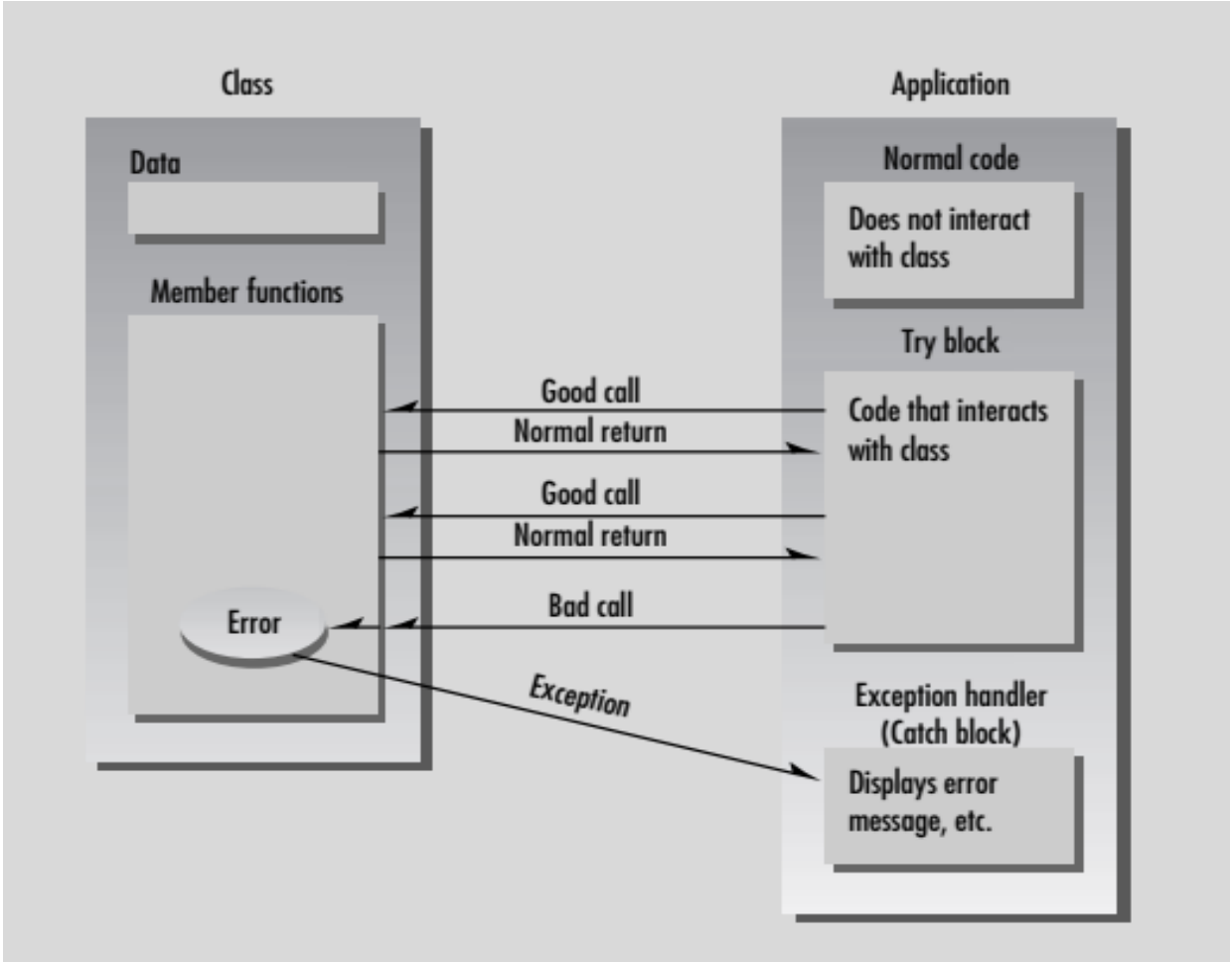
Output

```
Inside try block
Inside the function
Result = -3
Inside the function
Caught exception
```

Exception Class

- We can create our own exception class for throwing and catching.
- Throw or catch any C++ type that lives up to some simple requirements, namely that it has a valid copy constructor and destructor.
- Any code in the application that uses objects of the class is enclosed in a try block.
- Errors generated in the try block will be caught in the catch block.

Exception Class



Class type Example

```
#include <iostream>
using namespace std;
const int MAX = 3;
class Stack
{
private:
int st[MAX];
int top;
public:
Stack()
{
top = -1;
}
void push(int var)
{
if(top >= MAX-1)

{
throw Range();
}
else
st[++top] = var;
}
int pop()
{
if(top < 0)
throw Range();
else
return st[top--];
}
};
int main()
{
Stack s1;
try
{
s1.push(11);
s1.push(22);
s1.push(33);
s1.push(44);
cout << "1: " << s1.pop() << endl;
cout << "2: " << s1.pop() << endl;
cout << "3: " << s1.pop() << endl;
cout << "4: " << s1.pop() << endl;}
catch(Stack::Range())
{
cout << "Exception: Stack Full or Empty";
}
cout << "Arrive here after catch" ;
return 0;
}
```

Output

```
Exception: Stack Full or Empty
1: 11
2: 22
3: 33
Exception: Stack Full or Empty
Arrive here after catch
```

Handling Derived-Class Exceptions

- **catch** clause for a base class will also match any class derived from that base.
- Thus, if you want to catch exceptions of both a base class type and a derived class type, put the derived class first in the **catch** sequence.
- If you don't do this, only the base class **catch** will run.

Example

```
#include <iostream>
using namespace std;
class B
{
};
class D: public B
{
};
int main()
{
D derived;
try
{
throw derived;
}
catch(B b)
{
cout << "Caught a base class.\n";
}

catch(D d)
 {
cout << "This won't execute.\n";
}
return 0;
}
```

Output

```
Caught a base class.
```

Because **derived** is an object that has **B** as a base class, it will be caught by the first **catch** clause and the second clause will never execute. Some compilers will flag this condition with a warning message. Others may issue an error. Either way, to fix this condition, reverse the order of the **catch** clauses.

Exceptions with Arguments

- Throwing an exception involves not only transferring control to the handler, but also creating an object of the exception class by calling its constructor.

- Create an object of a class type when we throw the exception with the statement

```
throw class_name();
catch(class_name Object name)
{

}
```

Example

```
#include <iostream>
#include <string>
using namespace std;
class Distance
{
private:
int feet;
float inches;
public:
class InchesEx //exception class
{
public:
string origin;
float iValue;
InchesEx (string or, float in)
{
origin = or;
iValue = in;
} };
Distance()
{
feet = 0;
 inches = 0.0;
}
Distance(int ft, float in)
{
if(in >= 12.0)
throw InchesEx("2-arg constructor", in);
else
feet = ft;
inches = in;
}
void getdist()
{
cout << "\nEnter feet";
 cin >> feet;
cout << "Enter inches";
cin >> inches;
if(inches >= 12.0)
throw InchesEx ("getdist() function", inches);
}
void showdist()
{
cout << feet << "\'-" << inches << '\"'; }
};
int main()
{
try
{
Distance dist1(17, 3.5);
Distance dist2;
dist2.getdist();
cout << dist1.showdist();
cout << dist2.showdist();
}
catch(Distance::InchesEx ix)
{
cout << "\nInitialization error in " << ix.origin
<< ".\n Inches value of " << ix.iValue
<< " is too large.";
}
}
cout << endl;
return 0;
}
```

Rethrowing An Exception

- A handler may decide to rethrow the exception caught without processing it.

- In such a case we have to invoke **throw** without any arguments as shown below

```
throw;
```

↳ Preserve the exact same exception type that was originally caught.

- This causes the **current exception** to be thrown to the next enclosing **try/catch sequence** and is caught by a **catch statement listed after the enclosing try block.**

Example of "rethrowing" an exception.

```
#include <iostream>
using namespace std;
void Xhandler()
{
try
{
throw "hello"; // throw a char *
}
catch(const char *)
{
cout<<"Caught char * inside Xhandler\n";
throw ; // rethrow char * out of function
}
}
```

```
int main()
{
cout << “Main Start\n";
Try
{
Xhandler();
}
catch(const char *)
{
cout << "Caught char * inside main\n";
}
cout << “Main End";
return 0;
}
```

Output

```
Main start
Caught char * inside Xhandler
Caught char * inside main
Main end
```

Specifying Exception

- It is possible to restrict a function to throw certain exceptions by adding a **throw** list clause to the function definition.

```
type function(arg-list) throw(type-list)
{
… … …
… … …
… … …
}
```

Specifying Exception

- The type-list specifies the type of exception that may be thrown.
- Throwing any other kind of exception will cause abnormal program termination.
- If you want to prevent a function from throwing any exception, you may do so by making the type-list empty.

Example

```
#include<iostream>
using namespace std;
void test(int x) throw(char, double)
{
 if(x==0)
    throw 'x'; //char
 else
  if(x==1)
    throw x;  //int
  else
  if(x==-1)
    throw 1.0;  //double
}
int main()
{
Try
{
      cout<<"Testing throw restrictions \n";
      cout<<"x==0\n";
      test(0);
      cout<<"x==1\n";
      test(1);
      cout<<"x==-1\n";
      test(-1);
      cout<<"x==2\n";
      test(2);
}
catch (char c)
{
cout<<"caught a character\n";
}
 catch(int m)
{
 cout<<"caught an integer\n";
}
catch(double d)
{
cout<<"caught a double\n";}
return 0;}
```

Output

```
Testing throw restrictions
x==0
caught a character
x==2
caught a double
```

try-catch blocks can be nested.

```
#include <iostream>
using namespace std;
int main()
{
    try
{
        try
{
            throw 20;
        }
        catch (int n) {
            cout << "Handle Partially\n";
            throw;   //Re-throwing an exception
        }
    }
    catch (int n) {
        cout << "Handle remaining ";
                }
    return 0;
}
```

Output

Handle Partially
Handle remaining

Destructor with try catch

```
#include <iostream>
using namespace std;
 class Test {
public:
   Test() { cout << "Constructor of Test " << endl; }
  ~Test() { cout << "Destructor of Test "  << endl; }
};
 int main() {
  try {
    Test t1;
    throw 10;
  }
  catch(int i)
{
    cout << "Caught " << i << endl;
  }
}
```

Output

```
Output:
Constructor of Test
 Destructor of Test Caught 10
```

Destructor with try catch

```
#include <iostream>
using namespace std;
class Sample1
{
  public:
    Sample1()
{
cout << "Construct an Object of sample1“;
}
 ~Sample1()
{
       cout << "Destruct an Object of sample1;
    }
};
class Sample2
{
  public:
    Sample2() {
      int i =7;
      cout << "Construct an Object of sample2";
      throw i;
    }
    ~Sample2() {
      cout << "Destruct an Object of sample2";
    }
};
int main() {
  try {
    Sample1 s1;
    Sample2 s2;
  }
catch(int i) {
    cout << "Caught " << i << endl;
}
```

Output

```
Output:
Construct an Object of sample1
Construct an Object of sample2
Destruct an Object of sample1
Caught 7
```

Destructor with try catch

```
#include <iostream>
using namespace std;
 class Test
{
    static int count;
    int id;
 public:
Test()
{
count++;
id = count;
cout << "Constructing object number " << id << ;
if (id == 4)
throw 4;
    }
~Test()
    {
    cout << "Destructing object number " << id;
    }
};

int Test::count = 0;

int main()
{
    try
{
        Test array[5];
    }
    catch (int i)
{
        cout << "Caught " << i << endl;
    }
}
```

Output

```
Output:
Constructing object number 1
Constructing object number 2
Constructing object number 3
Constructing object number 4
Destructing object number 3
Destructing object number 2
Destructing object number 1
Caught 4
```

THANKS…..